INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Programación Web 2				
TÍTULO DE LA PRÁCTICA:	NodeJS + Express				
NÚMERO DE PRÁCTICA:	05	**AÑO LECTIVO:**	2026 - B	**NRO. SEMESTRE:**	III
FECHA DE PRESENTACIÓN	*21/10/2026*	**HORA DE PRESENTACIÓN**	*02:06 pm*		
INTEGRANTE (s) *DONNY MOISES MARA MAMANI*				**NOTA (0-20)**	
DOCENTE(s): *LINO JOSE PINTO OPPE*					

RESULTADOS Y PRUEBAS

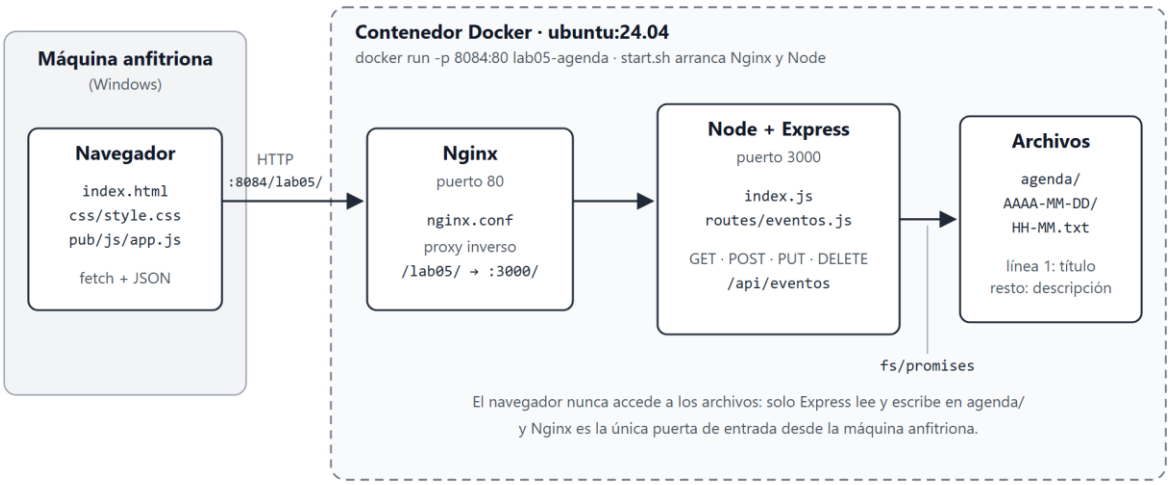
I. EJERCICIOS RESUELTOS:

Se desarrolló una aplicación web con **NodeJS** y **Express** para administrar una **agenda personal**, trabajando todo en una sola interfaz. Cada evento se guarda como un archivo de texto dentro de la carpeta `agenda/`, con una carpeta por fecha (`AAAA-MM-DD`) y un archivo por hora (`HH-MM.txt`); la primera línea del archivo es el título y las siguientes son la descripción. La aplicación permite crear, ver (en formato árbol), editar y eliminar eventos, y en producción se publica a través de **Nginx** dentro de un contenedor **Docker**.

Repositorio: https://github.com/LINOPINTO2023/Tutor-a-PW2-/tree/main/Mara_Mamani_Donny_Moises/LABORATORIO_05

1.1. Arquitectura de la solución

La página web no guarda ni lee los eventos por sí misma: se los pide al servidor mediante peticiones HTTP con `fetch`, y los datos viajan en formato JSON. El servidor de Express es el único que trabaja con los archivos de la carpeta `agenda/`, usando el módulo `fs/promises`, que permite leer y escribir archivos sin detener al servidor mientras espera. En producción, todas las visitas entran primero por Nginx, que está en el puerto 8084 de la computadora, y Nginx se las pasa a Express, que funciona dentro del contenedor en el puerto 3000.

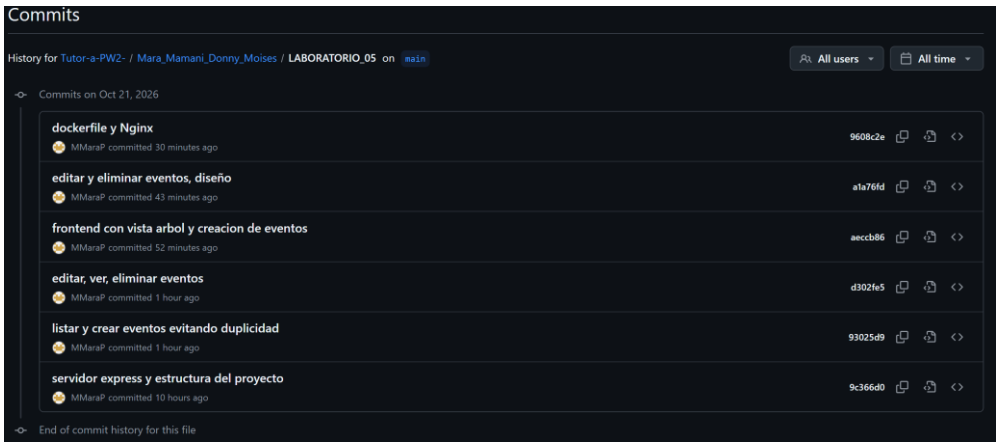


1.2. Estructura de directorios

Estructura del laboratorio hasta el nivel 4:

```
LABORATORIO_05/
├── .dockerignore
├── .gitattributes
├── .gitignore
├── Dockerfile
├── README.md
├── agenda/
│   └── 2026-10-28/
│       └── 16-31.txt
├── css/
│   └── style.css
├── index.html
├── index.js
├── nginx.conf
├── package-lock.json
├── package.json
├── pub/
│   └── js/
│       └── app.js
├── routes/
│   └── eventos.js
└── start.sh
```

1.3. Commits



Se creó el proyecto con `npm init`, se instaló Express y se escribió el servidor base en `index.js`. Este archivo configura tres cosas: que el servidor entienda los datos en formato JSON que envía la página (`express.json()`), que publique los archivos del cliente como el CSS y el JavaScript (`express.static`), y que al entrar a la dirección principal `/` se muestre `index.html`. El servidor escucha en el puerto 3000, salvo que se indique otro con la variable `PORT`. Las rutas de los eventos se conectan con la línea `app.use('/api/eventos', eventosRouter)`.

index.js

```
const path = require('path');
const express = require('express');
const eventosRouter = require('./routes/eventos');

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use('/css', express.static(path.join(__dirname, 'css')));
app.use(express.static(path.join(__dirname, 'pub')));

app.get('/', (request, response) => {
    response.sendFile(path.resolve(__dirname, 'index.html'));
});

app.use('/api/eventos', eventosRouter);

app.listen(PORT, () => {
    console.log(`Escuchando en: http://localhost:${PORT}`);
});
```

Las rutas de eventos se separaron en `routes/eventos.js` usando `express.Router()`, de modo que `index.js` solo configura el servidor. Antes de guardar un evento, el servidor revisa que los datos sean correctos. La fecha debe tener el formato `AAAA-MM-DD` y la hora `HH-MM`; esto se comprueba con expresiones regulares. Además de evitar datos mal escritos, impide que alguien envíe una ruta extraña para guardar archivos fuera de la carpeta

agenda/. El título es obligatorio y debe ocupar una sola línea, ya que la primera línea del archivo se usa como título.

routes/eventos.js — validaciones

```
const AGENDA_DIR = path.join(__dirname, '..', 'agenda');
const FECHA_RE = /^\d{4}-\d{2}-\d{2}$/;      // 2026-10-21
const HORA_RE = /^\d{2}-\d{2}$/;             // 10-30

function fechaHoraValidas(fecha, hora) {
    return FECHA_RE.test(fecha) && HORA_RE.test(hora);
}

function tituloValido(titulo) {
    return typeof titulo === 'string' && titulo.trim() !== '' && !/[\r\n]/.test(titulo);
}

function contenidoEvento(titulo, descripcion = '') {
    return `${titulo.trim()}\n${descripcion}`;
}
```

Para crear un evento, el archivo se guarda con la opción `'wx'`, que solo escribe el archivo si todavía no existe. Si ya hay un evento en esa fecha y hora, Node da el error `EEXIST` y el servidor responde con el código **409 (conflicto)** y un mensaje de error. Así se cumple el requisito de no registrar un evento cuando el archivo ya existe, sin riesgo de borrar el que estaba antes.

routes/eventos.js — POST /api/eventos

```
router.post('/', async (request, response) => {
    const { fecha, hora, titulo, descripcion } = request.body;

    if (!fechaHoraValidas(fecha, hora)) {
        return response.status(400).json({ error: ERROR_FECHA_HORA });
    }
    if (!tituloValido(titulo)) {
        return response.status(400).json({ error: ERROR_TITULO });
    }

    try {
        await fs.mkdir(path.join(AGENDA_DIR, fecha), { recursive: true });
        await fs.writeFile(rutaEvento(fecha, hora), contenidoEvento(titulo, descripcion), {
flag: 'wx' });
        response.status(201).json({ mensaje: 'Evento creado' });
    } catch (err) {
        if (err.code === 'EEXIST') {
            return response.status(409).json({ error: 'Ya existe un evento en esa fecha y hora'
});
        }
        console.error(err);
        response.status(500).json({ error: 'No se pudo crear el evento' });
    }
});
```

Para mostrar el árbol, la ruta `GET /api/eventos` recorre las carpetas de cada fecha, abre cada archivo y toma solo su primera línea, que es el título. Devuelve la lista ordenada por fecha y hora, lista para que la página la dibuje.

```
routes/eventos.js — GET /api/eventos
const fechas = carpetas
    .filter(d => d.isDirectory() && FECHA_RE.test(d.name))
    .map(d => d.name)
    .sort();

const arbol = [];
for (const fecha of fechas) {
    const archivos = (await fs.readdir(path.join(AGENDA_DIR, fecha)))
        .filter(a => ARCHIVO_RE.test(a))
        .sort();

    const eventos = [];
    for (const archivo of archivos) {
        const contenido = await fs.readFile(path.join(AGENDA_DIR, fecha, archivo), 'utf8');
        eventos.push({
            hora: archivo.replace('.txt', ''),
            titulo: contenido.split(/\r?\n/)[0]
        });
    }
    arbol.push({ fecha, eventos });
}
response.json(arbol);
```

Para trabajar con un evento en particular, su fecha y hora se indican en la dirección. Por ejemplo, `/api/eventos/2026-10-21/10-00` corresponde al archivo `agenda/2026-10-21/10-00.txt`. La ruta de detalle devuelve el título y la descripción completos para mostrarlos al editar. Al editar se reescribe el archivo con los nuevos datos; la fecha y la hora no se pueden cambiar porque son el nombre del archivo. Si el evento no existe, el servidor responde con el código 404.

```
routes/eventos.js — GET y PUT /api/eventos/:fecha/:hora
router.get('/:fecha/:hora', async (request, response) => {
    ...
    const contenido = await fs.readFile(rutaEvento(fecha, hora), 'utf8');
    const [titulo, ...resto] = contenido.split(/\r?\n/);
    response.json({ fecha, hora, titulo, descripcion: resto.join('\n') });
    ...
});

router.put('/:fecha/:hora', async (request, response) => {
    ...
    await fs.access(rutaEvento(fecha, hora));
    await fs.writeFile(rutaEvento(fecha, hora), contenidoEvento(titulo, descripcion));
    response.json({ mensaje: 'Evento actualizado' });
    ...
});
```

Al **eliminar**, si la carpeta del día queda vacía también se borra, para que el árbol no muestre fechas sin eventos:

```
routes/eventos.js — DELETE /api/eventos/:fecha/:hora
router.delete('/:fecha/:hora', async (request, response) => {
    const { fecha, hora } = request.params;

    if (!fechaHoraValidas(fecha, hora)) {
```

```
        return response.status(400).json({ error: ERROR_FECHA_HORA });
    }

    try {
        await fs.unlink(rutaEvento(fecha, hora));

        const carpeta = path.join(AGENDA_DIR, fecha);
        if ((await fs.readdir(carpeta)).length === 0) {
            await fs.rmdir(carpeta);
        }
        response.json({ mensaje: 'Evento eliminado' });
    } catch (err) {
        if (err.code === 'ENOENT') {
            return response.status(404).json({ error: ERROR_NO_EXISTE });
        }
        console.error(err);
        response.status(500).json({ error: 'No se pudo eliminar el evento' });
    }
});
```

De esta forma la API sigue el estilo **REST**: cada acción usa un método HTTP distinto (GET para consultar, POST para crear, PUT para modificar y DELETE para eliminar) y cada respuesta lleva un código que indica si la operación salió bien o qué problema hubo.

La interfaz es una sola página: el formulario a la izquierda y el árbol de eventos a la derecha, con el formato de carpetas y archivos que pide el enunciado. La página se comunica con el servidor usando `fetch` y `async/await`, por lo que nunca necesita recargarse. Todas las peticiones pasan por la función `pedir()`, que envía los datos, recibe la respuesta y, si algo falla, muestra el mensaje de error que devolvió el servidor.

pub/js/app.js — función pedir()

```
const API = 'api/eventos';

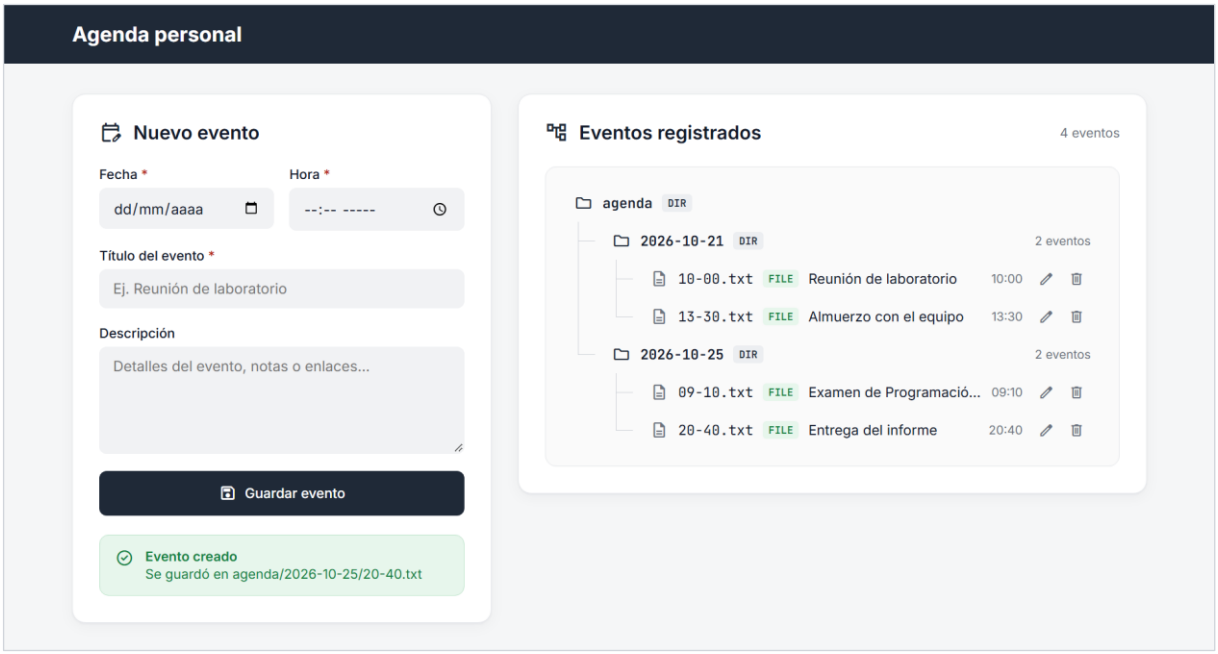
async function pedir(url, opciones = {}) {
    let response;
    try {
        response = await fetch(url, {
            headers: { 'Content-Type': 'application/json' },
            ...opciones
        });
    } catch {
        throw new Error('No se pudo conectar con el servidor');
    }
    const data = await response.json().catch(() => ({}));
    if (!response.ok) {
        const error = new Error(data.error || `Error ${response.status}`);
        error.status = response.status;
        throw error;
    }
    return data;
}
```

Las direcciones que usa `fetch` se escribieron de forma relativa (`api/eventos` en lugar de `/api/eventos`), para que la aplicación funcione tanto en `localhost:3000` como detrás de Nginx en `/lab05/`. Además, el árbol se

arma con `textContent`, así que si alguien escribe código HTML en un título, se muestra como texto normal y no se ejecuta.

```
pub/js/app.js — construcción segura del árbol
function crearElemento(etiqueta, clase, texto = '') {
    const elemento = document.createElement(etiqueta);
    elemento.className = clase;
    elemento.textContent = texto;
    return elemento;
}

async function cargarEventos() {
    try {
        dibujarArbol(await pedir(API));
    } catch (err) {
        mostrarMensaje('error', 'No se pudo cargar la agenda', err.message);
    }
}
```



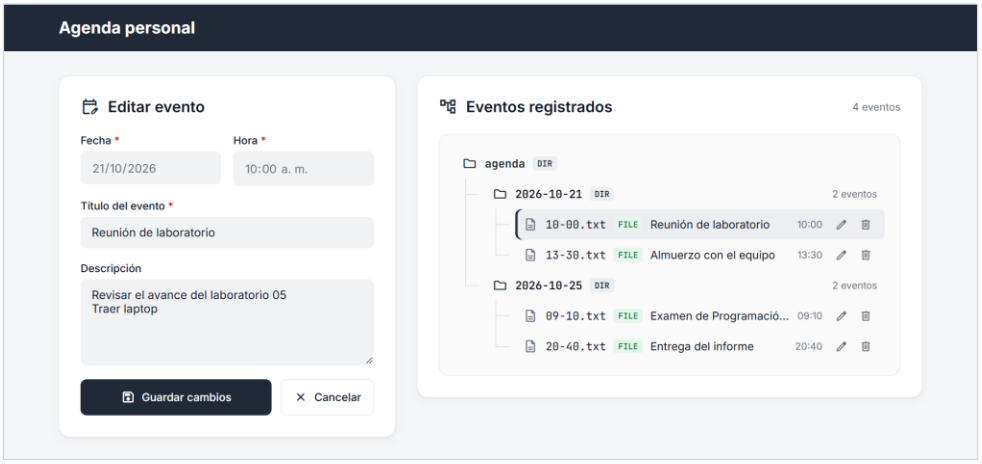
Cada evento del árbol tiene un botón para editar y otro para eliminar. Al pulsar editar, la página pide al servidor el contenido del archivo y llena el formulario con él; la fecha y la hora quedan bloqueadas y el evento se resalta en el árbol. Al guardar, si se está editando se envía una petición PUT, y si se está creando un evento nuevo, una petición POST.

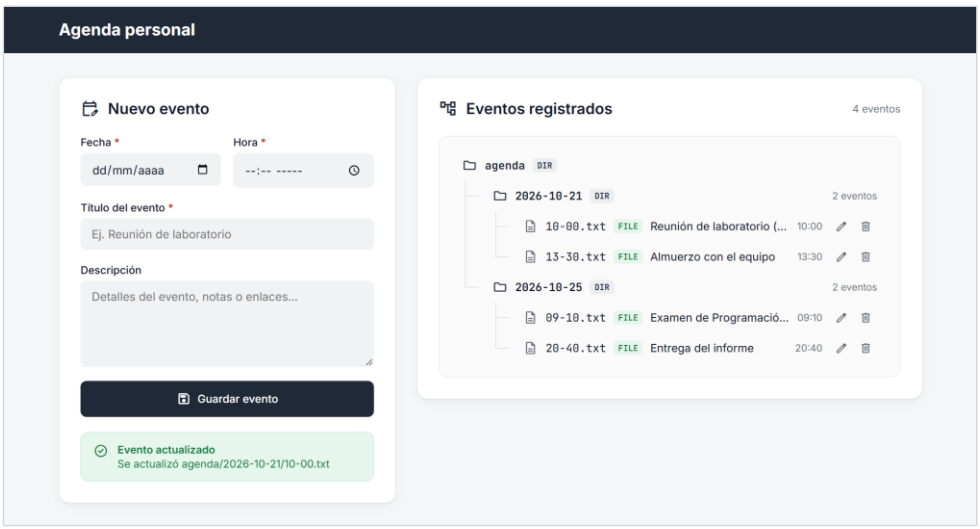
```
pub/js/app.js — modo edición
async function editarEvento(fecha, hora) {
    try {
        const evento = await pedir(rutaEvento(fecha, hora));
        editando = { fecha, hora };
        campoFecha.value = fecha;
```

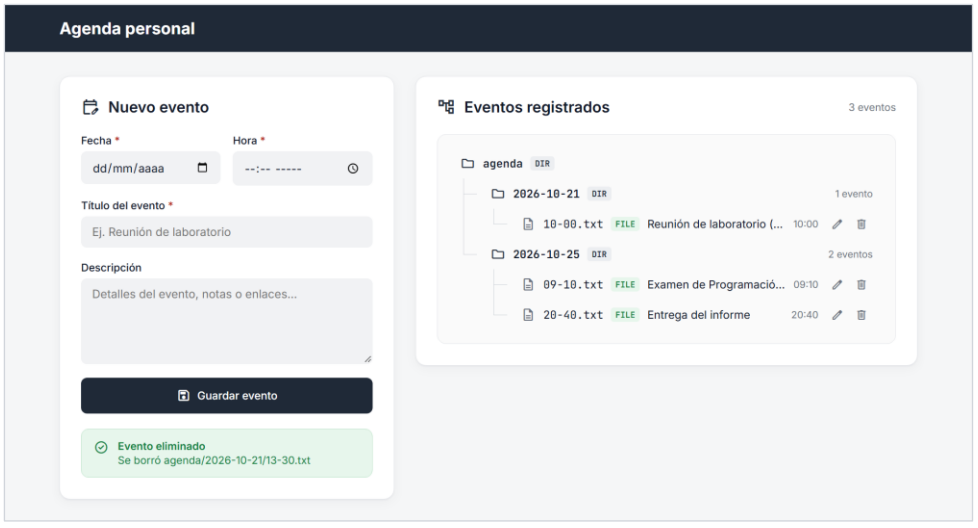
```
        campoHora.value = horaLegible(hora);
        campoTitulo.value = evento.titulo;
        campoDescripcion.value = evento.descripcion;
        campoFecha.disabled = true;
        campoHora.disabled = true;
        tituloFormulario.textContent = 'Editar evento';
        textoGuardar.textContent = 'Guardar cambios';
        btnCancelar.hidden = false;
        mensaje.hidden = true;
        marcarEditando();
        campoTitulo.focus();
    } catch (err) {
        mostrarMensaje('error', err.message);
        await cargarEventos();
    }
}
```

pub/js/app.js — envío del formulario (PUT o POST)

```
if (editando) {
    const respuesta = await pedir(rutaEvento(editando.fecha, editando.hora), {
        method: 'PUT',
        body: JSON.stringify(contenido)
    });
    modoCreacion();
    mostrarMensaje('exito', respuesta.mensaje, `Se actualizó ${archivo}`);
} else {
    const respuesta = await pedir(API, {
        method: 'POST',
        body: JSON.stringify({ fecha, hora, ...contenido })
    });
    form.reset();
    mostrarMensaje('exito', respuesta.mensaje, `Se guardó en ${archivo}`);
}
await cargarEventos();
```


Agenda personal
Editar evento
Fecha *
21/10/2026
Hora *
10:00 a. m.
Título del evento *
Reunión de laboratorio
Descripción
Revisar el avance del laboratorio 05
Traer laptop
Guardar cambios
Cancelar
Eventos registrados
4 eventos
agenda DIR
2026-10-21 DIR
2 eventos
10-00.txt FILE Reunión de laboratorio 10:00
13-30.txt FILE Almuerzo con el equipo 13:30
2026-10-25 DIR
2 eventos
09-10.txt FILE Examen de Programació... 09:10
20-40.txt FILE Entrega del informe 20:40


Agenda personal
Nuevo evento
Fecha *
dd/mm/aaaa
Hora *
--:-- -----
Título del evento *
Ej. Reunión de laboratorio
Descripción
Detalles del evento, notas o enlaces...
Guardar evento
Evento actualizado
Se actualizó agenda/2026-10-21/10-00.txt
Eventos registrados
4 eventos
agenda DIR
2026-10-21 DIR
2 eventos
10-00.txt FILE Reunión de laboratorio (... 10:00
13-30.txt FILE Almuerzo con el equipo 13:30
2026-10-25 DIR
2 eventos
09-10.txt FILE Examen de Programació... 09:10
20-40.txt FILE Entrega del informe 20:40


Agenda personal
Nuevo evento
Fecha *
dd/mm/aaaa
Hora *
--:-- -----
Título del evento *
Ej. Reunión de laboratorio
Descripción
Detalles del evento, notas o enlaces...
Guardar evento
Evento eliminado
Se borró agenda/2026-10-21/13-30.txt
Eventos registrados
3 eventos
agenda DIR
2026-10-21 DIR
1 evento
10-00.txt FILE Reunión de laboratorio (... 10:00
2026-10-25 DIR
2 eventos
09-10.txt FILE Examen de Programació... 09:10
20-40.txt FILE Entrega del informe 20:40

Para producción se creó un `Dockerfile`, que es la receta que usa Docker para construir la imagen. Parte de **Ubuntu 24.04**, instala Nginx, NodeJS y npm con `apt-get`, instala las dependencias del proyecto y copia los archivos de la aplicación junto con la configuración de Nginx. Se eligió Ubuntu 24.04 porque su versión de NodeJS es compatible con Express 5; versiones anteriores de Ubuntu instalan un NodeJS demasiado antiguo.

```
Dockerfile
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
    && apt-get install -y --no-install-recommends nginx nodejs npm \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY . .
COPY nginx.conf /etc/nginx/sites-available/default
RUN chmod +x start.sh

EXPOSE 80

CMD ["./start.sh"]
```

Nginx funciona como **proxy inverso**, es decir, como intermediario entre el navegador y la aplicación. Recibe las visitas en el puerto 80 del contenedor y las que llegan a `/lab05/` las envía a Express, que está en el puerto 3000. De esta forma, el usuario nunca se conecta directamente con Node, sino siempre a través de Nginx. Si alguien entra a `/` o a `/lab05` sin la barra final, Nginx lo redirige automáticamente a `/lab05/`.

```
nginx.conf
server {
    listen 80;
    server_name _;
    absolute_redirect off;

    location = / {
        return 302 /lab05/;
    }

    location = /lab05 {
        return 301 /lab05/;
    }

    location /lab05/ {
        proxy_pass http://127.0.0.1:3000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

```
}
```

Un contenedor normalmente ejecuta un solo programa, pero esta aplicación necesita dos: Nginx y Node. Por eso se creó el script `start.sh`, que primero inicia Nginx y luego la aplicación de Node. Además, el archivo `.dockerignore` indica qué no se debe copiar al contenedor, como `node_modules` o los eventos guardados en la computadora local.

start.sh

```
#!/bin/sh
set -e

nginx
exec node index.js
```

II. PRUEBAS

2.1. Ejecución en local

Requisitos: NodeJS 18 o superior. Desde la carpeta del laboratorio:

```
$ npm install
$ node index.js
Escuchando en: http://localhost:3000
```

Se probó crear un evento, intentar crear un duplicado, listar, editar y eliminar usando `curl`. Debajo de cada comando se muestra la respuesta obtenida:

```
$ curl -i -X POST localhost:3000/api/eventos -H "Content-Type: application/json" \
       -d '{"fecha":"2026-10-21","hora":"10-00","titulo":"Reunion de laboratorio"}'
HTTP/1.1 201 Created
{"mensaje":"Evento creado"}

$ curl -i -X POST localhost:3000/api/eventos -H "Content-Type: application/json" \
       -d '{"fecha":"2026-10-21","hora":"10-00","titulo":"Otro"}'
HTTP/1.1 409 Conflict
{"error":"Ya existe un evento en esa fecha y hora"}

$ curl localhost:3000/api/eventos
[{"fecha":"2026-10-21","eventos":[{"hora":"10-00","titulo":"Reunion de laboratorio"}]}]

$ curl -i -X PUT localhost:3000/api/eventos/2026-10-21/10-00 -H "Content-Type: application/json"
\
       -d '{"titulo":"Reunion cambiada","descripcion":"Nueva descripcion"}'
HTTP/1.1 200 OK
{"mensaje":"Evento actualizado"}

$ curl -i -X DELETE localhost:3000/api/eventos/2026-10-21/10-00
HTTP/1.1 200 OK
{"mensaje":"Evento eliminado"}
```

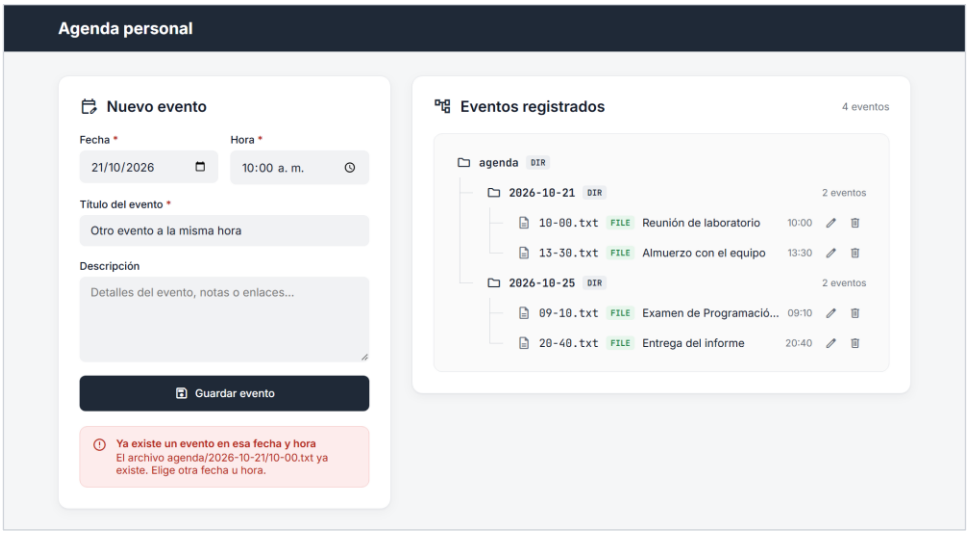
2.2. Ejecución en producción con Docker y Nginx

```
$ docker build -t lab05-agenda .
$ docker run -d --name lab05 -p 8084:80 lab05-agenda
$ docker ps
$ docker exec lab05 sh -c 'node -v; nginx -v'
v18.19.1
nginx version: nginx/1.24.0 (Ubuntu)

$ curl -I 127.0.0.1:8084/lab05/
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Content-Type: text/html; charset=utf-8

$ curl -s -o /dev/null -w "%{http_code} %{redirect_url}" 127.0.0.1:8084/lab05
301 http://127.0.0.1:8084/lab05/
```

En la respuesta aparece `Server: nginx/1.24.0 (Ubuntu)`, lo que confirma que la petición pasó por Nginx antes de llegar a la aplicación.



III. CUESTIONARIO:

Pregunta: Mencione la diferencia entre conexiones asíncronas usando el objeto XmlHttpRequest, JQuery.ajax y Fetch. Justifique su respuesta con un ejemplo muy básico.

Las tres permiten hacer peticiones HTTP sin recargar la página (AJAX); la diferencia está en la forma de escribirlas y de manejar la respuesta:

Ejemplo "Hola mundo". Se tiene una ruta en Express que responde un JSON, y en el cliente un elemento `<p id="resultado">` donde se mostrará el mensaje. La misma petición se hace de las tres formas:

```
Servidor (Express)
app.get('/saludo', (request, response) => {
```

```
    response.json({ mensaje: 'Hola mundo' });
});
```

Cliente con XMLHttpRequest

```
function saludarXHR() {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', 'saludo');
    xhr.onload = () => {
        if (xhr.status === 200) {
            const data = JSON.parse(xhr.responseText);
            document.querySelector('#resultado').textContent = data.mensaje;
        } else {
            console.error('Error', xhr.status);
        }
    };
    xhr.onerror = () => console.error('Error de red');
    xhr.send();
}
```

Cliente con jQuery.ajax

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

function saludarJQuery() {
    $.ajax({ url: 'saludo', method: 'GET', dataType: 'json' })
        .done(data => $('#resultado').text(data.mensaje))
        .fail((xhr, estado) => console.error('Error', xhr.status, estado));
}
```

Cliente con Fetch

```
async function saludarFetch() {
    try {
        const response = await fetch('saludo');
        if (!response.ok) {
            throw new Error(`Error ${response.status}`);
        }
        const data = await response.json();
        document.querySelector('#resultado').textContent = data.mensaje;
    } catch (err) {
        console.error(err.message);
    }
}
```

Las tres muestran "Hola mundo" en la página. Con **XMLHttpRequest** hay que crear el objeto, abrir la conexión, registrar los eventos y convertir el JSON a mano. **jQuery.ajax** simplifica la sintaxis y convierte el JSON automáticamente, pero obliga a cargar una librería externa. **Fetch** es nativo, se basa en promesas y con `async/await` el código se lee de forma secuencial; su particularidad es que una respuesta 404 o 500 no se considera error, por lo que hay que revisar `response.ok`.

CONCLUSIONES

- Node.js permite usar JavaScript en el servidor con un modelo asíncrono y no bloqueante. Al usar `fs/promises` con `async/await`, el servidor puede atender otras peticiones mientras lee o escribe los archivos de la agenda.
- Express facilita construir una API REST ordenada: con `express.Router()` las rutas de eventos quedaron separadas del servidor, y los verbos HTTP junto con los códigos de estado (201, 400, 404, 409) comunican claramente el resultado de cada operación.
- Docker y Nginx permitieron reproducir un entorno de producción con solo dos comandos. Nginx como proxy inverso es la única puerta de entrada, y la aplicación de Node queda protegida detrás de él.
- Dividir el trabajo en commits pequeños (primero la API, luego la interfaz y al final el despliegue) permitió probar cada parte por separado y muestra la evolución del código en el historial del repositorio.

METODOLOGÍA DE TRABAJO

1. **Análisis del enunciado:** se identificaron los requisitos de la agenda (crear sin duplicados, ver en árbol, editar, eliminar), el formato de almacenamiento y el despliegue con Docker y Nginx.
2. **Servidor base:** se creó el proyecto con `npm init`, se instaló Express y se configuró el servidor y la estructura de carpetas.
3. **API REST:** primero listar y crear con validaciones y rechazo de duplicados; después ver detalle, editar y eliminar. Cada ruta se probó con `curl`, incluyendo los casos de error.
4. **Interfaz:** se construyó la página única con el formulario y el árbol usando `fetch`, y luego el modo edición y la eliminación. Se probó en el navegador cada flujo.
5. **Despliegue:** se escribió el Dockerfile con Ubuntu 24.04, la configuración de Nginx y el script de arranque, y se verificó la aplicación en http://127.0.0.1:8084/lab05/.
6. **Documentación:** se redactó el README con las instrucciones de ejecución y este informe con las evidencias, pruebas y la respuesta al cuestionario.

REFERENCIAS Y BIBLIOGRAFÍA

[1] R. S. Escobedo Quispe, "backend-js," GitHub. [En línea]. Disponible en: https://github.com/rescobedoq/backend-js

[2] OpenJS Foundation, "Node.js v18 Documentation," 2024. [En línea]. Disponible en: https://nodejs.org/docs/latest-v18.x/api/

[3] Express, "Express: Infraestructura web rápida, minimalista y flexible para Node.js." [En línea]. Disponible en: https://expressjs.com/es/

[4] W3Schools, "Node.js Introduction." [En línea]. Disponible en: https://www.w3schools.com/nodejs/nodejs_intro.asp

[5] Docker Inc., "Dockerfile reference," Docker Docs. [En línea]. Disponible en: https://docs.docker.com/reference/dockerfile/

[6] F5 Inc., "NGINX Reverse Proxy," NGINX Documentation. [En línea]. Disponible en: https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

ANEXOS

A. Rúbrica de autoevaluación

Según la sección 5.2 del enunciado, el estudiante marca la columna Checklist y se autocalifica en la columna Estudiante (2.0 o 4.0 puntos según el nivel de desempeño: 25 %, 50 %, 75 % o 100 %).

Criterio	Descripción	Puntos	Checklist	Estudiante	Profesor
1. GitHub	Repositorio se pudo clonar y se evidencia la estructura adecuada para revisar los entregables.	4	✔	3	
2. Commits	Hay porciones de código fuente asociado a los commits planificados con explicaciones detalladas.	4	✔	3	
3. Ejecución	Se incluyen comandos para ejecuciones y pruebas del código fuente explicadas gradualmente que permitirían replicar el proyecto.	4	✔	4	
4. Pregunta	Se responde con completitud a la pregunta formulada en la tarea.	2	✔	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	✔	2	
8. Madurez	El informe muestra la evolución de la madurez del código fuente con explicaciones puntuales pero precisas, agregando diagramas generados a partir del código fuente.	4	✔	3	
Total		20		16	